



UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING

Table of Contents

INTRODUCTION	3
LEARNING OBJECTIVES.....	3
1.1 MACHINE LEARNING OVERVIEW, BASIC PRINCIPLES	4
1.1.1 WHAT IS MACHINE LEARNING?	4
1.1.2 HISTORICAL CONTEXT AND EVOLUTION.....	4
1.1.3 COMPONENTS OF MACHINE LEARNING SYSTEMS	6
1.1.4 THE PROMISE OF QUANTUM MACHINE LEARNING	7
1.2 TYPES OF LEARNING, APPLICATIONS	8
1.2.1 SUPERVISED LEARNING.....	8
1.2.2 UNSUPERVISED LEARNING	9
1.2.3 REINFORCEMENT LEARNING	11
1.2.4 THE FOUR SECTORS OF QML	11
1.2.5 INTRODUCTION TO KERNEL METHODS	13
1.2.6 APPLICATIONS OF MACHINE LEARNING	15
1.3 SCI-KIT LEARN FRAMEWORK.....	16
1.3.1 INTRODUCTION TO SCIKIT-LEARN	16
1.3.2 DATA HANDLING IN SCIKIT-LEARN.....	17
1.3.3 BUILDING YOUR FIRST MODEL	18



UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING

1.3.4 MODEL EVALUATION	20
1.3.5 PRACTICAL EXAMPLE: CLASSIFICATION TASK	21
1.4 SUMMARY	23
1.5 MIND MAP	24
1.6 GLOSSARY.....	24
1.7 SELF-ASSESSMENT QUESTIONS	26
1.8 TERMINAL QUESTIONS	28
1.9 ANSWERS	29
1.9.1 SELF-ASSESSMENT ANSWERS	29
1.9.2 TERMINAL QUESTION ANSWERS	31
1.10 REFERENCES	35



INTRODUCTION

This unit introduces the foundational concepts of classical machine learning that serve as essential prerequisites for understanding quantum machine learning. Machine learning has revolutionized how we process information and make predictions from data, and understanding these classical foundations is crucial before exploring their quantum counterparts.

In this unit, you will learn about the basic principles of machine learning, explore different types of learning paradigms, understand the mathematical concept of kernel methods, and gain hands-on experience with the Scikit-learn framework for implementing ML models.

Before studying this unit, you should have:

- Basic programming knowledge (preferably Python)
- Understanding of basic mathematics (algebra, elementary calculus)
- Familiarity with data structures (arrays, matrices)

LEARNING OBJECTIVES

Upon successful completion of this unit, you will be able to:

1. **LO1:** Explain machine learning fundamentals and distinguish ML from traditional programming
2. **LO2:** Classify different types of learning paradigms (supervised, unsupervised, reinforcement learning)
3. **LO3:** Apply the kernel trick concept to understand how non-linear data can be handled
4. **LO4:** Implement basic ML models using the Scikit-learn framework



1.1 MACHINE LEARNING OVERVIEW, BASIC PRINCIPLES

1.1.1 WHAT IS MACHINE LEARNING?

Definition: Machine learning is a branch of artificial intelligence that focuses on building systems capable of learning from data, identifying patterns, and making decisions with minimal human intervention.

Unlike traditional programming where developers explicitly code rules to solve problems, machine learning systems learn these rules from examples. Consider the difference:

Traditional Programming	Machine Learning
Input: Data + Rules	Input: Data + Expected Output
Output: Answer	Output: Rules (Model)
Explicit instructions for every scenario	Learns patterns from examples
Static behaviour	Adaptive behaviour

Table 1.1: Traditional programming versus machine learning.

The Learning Process

The machine learning process follows a fundamental pattern:

1. **Data Collection:** Gathering relevant information
2. **Data Preparation:** Cleaning and formatting data
3. **Model Selection:** Choosing an appropriate algorithm
4. **Training:** The model learns patterns from data
5. **Evaluation:** Testing model performance
6. **Prediction:** Using the model on new data

This iterative process allows systems to improve their performance over time as they are exposed to more data.

1.1.2 HISTORICAL CONTEXT AND EVOLUTION

The Computing Revolution



The advancement of computational power has always been a driving force behind every major industrial revolution. The invention of the modern computer, followed by the central processing unit (CPU), led to the "digital revolution," transforming industries through process automation and the rise of information technology.

More recently, the development of graphical processing units (GPUs) has powered the era of artificial intelligence and big data, enabling breakthroughs in areas such as:

- Intelligent transportation systems
- Autonomous driving
- Scientific simulations
- Complex data analysis

The Limits of Moore's Law

However, as we approach the physical and practical limits of Moore's law—the principle that the number of transistors on a chip doubles approximately every two years—traditional computing devices like CPUs and GPUs are nearing the end of their scaling potential.

The ever-growing demand for computational power, driven by the exponential increase in data and the complexity of modern applications, necessitates new computing paradigms. Among the leading candidates to meet these challenges are **quantum computers**, which leverage the unique principles of quantum mechanics such as superposition and entanglement to process information in ways that classical systems cannot.

The Emergence of Quantum Machine Learning

The convergence of the computational power offered by quantum machines and the limitations faced by AI models has led to the rapid emergence of the field: **quantum machine learning (QML)**.

The challenges in modern AI stem from the neural scaling law, which posits that "bigger is often better." Since 2020, this principle has driven the development of increasingly colossal models, featuring more complex architectures and an ever-growing number of parameters. However, this progress comes at an immense cost. For instance, training a



model like ChatGPT on a single GPU would take approximately 355 years, while the cloud computing costs for training such large models can reach tens of thousands of dollars.

Quantum computing, celebrated for its extraordinary computational capabilities, holds the potential to overcome these limitations. It offers the possibility of advancing models like generative pre-trained transformers (GPTs) and accelerating progress toward artificial general intelligence (AGI).

1.1.3 COMPONENTS OF MACHINE LEARNING SYSTEMS

Both classical and quantum computing systems share three fundamental components: **input**, **computation**, and **output**. Understanding these components helps illustrate how quantum computing differs from—and potentially improves upon—classical approaches.

[IMAGE: Figure 1.1 - Classical Computing VS Quantum Computing]

Source: Du et al., "Quantum Machine Learning: A Hands-on Tutorial," 2025, p. 4

Component	Classical	Quantum
Input	Binary bits	Quantum bits
Computation	Digital logical circuits	Quantum circuits
Output	Retrieve solution	Quantum measurements

Table 1.2: Comparison between classical and quantum computing.

Bits versus Qubits

A classical bit is a binary unit that takes on a value of either 0 or 1. In contrast, a quantum bit, or qubit, can exist in a superposition of both 0 and 1 simultaneously, represented by a two-dimensional vector where the entries correspond to the probabilities of the qubit being in each state.

Furthermore, while classical bits follow the Cartesian product rule, qubits adhere to the tensor product rule. This distinction implies that an N-qubit system is described by a 2^N -dimensional vector, allowing quantum systems to encode information exponentially with N—far surpassing the capacity of classical bits.



Classical	Quantum
Single bit ($N = 1$)	$x \in \{0, 1\}$
Multiple bits ($N > 1$)	$x \in \{0, 1\}^N$

Table 1.3: Mathematical representations of N -quantum bits.

The Quantum Advantage in Computation

Digital logic circuits versus quantum circuits: Classical computers rely on digital logic circuits composed of logic gates that perform operations on bits in a deterministic manner. In contrast, quantum circuits consist of **quantum gates**, which act on single or multiple qubits to modify their states—the probability amplitudes. Owing to the universality of quantum gates, for any given input qubit state, there always exists a specific quantum circuit capable of transforming the input state into one corresponding to the target solution.

For certain probability distributions, a quantum computer can use a polynomial number of quantum gates relative to the qubit count N to generate the distribution, whereas classical computers require an exponential number of gates with N to achieve the same result. This difference underpins the quantum advantage.

1.1.4 THE PROMISE OF QUANTUM MACHINE LEARNING

What is Quantum Machine Learning?

In its simplest terms, the focus on QML can be summarized as follows:

Quantum machine learning (informal): QML explores learning algorithms that can be executed on quantum computers to accomplish specified tasks with potential advantages over classical implementations.

The three key elements in this interpretation are: *quantum processors, specified tasks, and advantages.*

Understanding Quantum Advantage

Broadly, quantum advantage is demonstrated when quantum computers can solve a problem *more efficiently* than classical computers. However, the notion of "efficiency" is not uniquely defined.



1. **Runtime Complexity:** The most common measure. Quantum advantage is achieved when the upper bound of a quantum algorithm's runtime for a given task is *lower than* the theoretical lower bound of all possible classical algorithms for the same task.
2. **Sample Complexity:** Particularly within the Probably Approximately Correct (PAC) learning framework. Sample complexity is defined as the number of interactions (e.g., queries of target quantum systems or measurements) required for a learner to achieve a desired prediction accuracy below a specified threshold.
3. **Quantum Query Complexity:** Within frameworks of quantum statistical learning and quantum exact learning.

Quantum Utility

Quantum utility refers to quantum computations that yield reliable, accurate solutions to problems beyond the reach of brute-force classical methods and otherwise accessible only through classical approximation techniques. Reaching quantum utility signifies that quantum computers have attained a level of scale and reliability enabling researchers to use them as effective tools for scientific exploration, potentially leading to groundbreaking new insights.

1.2 TYPES OF LEARNING, APPLICATIONS

1.2.1 SUPERVISED LEARNING

Definition: Supervised learning is a type of machine learning where the algorithm learns from labelled training data to make predictions about unseen data.

In supervised learning, each training example consists of:

- **Input features (X):** The attributes or characteristics of the data
- **Target label (y):** The known correct output

The algorithm learns a mapping function $f: X \rightarrow y$ that can predict labels for new, unseen inputs.



Classification

Classification involves predicting categorical labels. Examples include:

- Email spam detection (spam or not spam)
- Image recognition (cat, dog, bird, etc.)
- Medical diagnosis (disease present or absent)

In classification, the goal is to find a decision boundary that best separates different classes of data. When the data is linearly separable, this boundary can be represented as a straight line (in 2D), a plane (in 3D), or a hyperplane (in higher dimensions).

Mathematically, given an input space $X \subset \mathbb{R}^d$ with $d \geq 1$ and a target or output space $Y = \{+1, -1\}$, we consider a training dataset $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n \in (X \times Y)^n$ where each data point $x^{(i)} \in X$ is associated with a label $y^{(i)} \in Y$. For the dataset to be linearly separable, there must exist a vector $w \in \mathbb{R}^d$ and a bias term $b \in \mathbb{R}$ such that:

$$\forall i \in [n], y^{(i)}(w^T x^{(i)} + b) \geq 0$$

Regression

Regression involves predicting continuous numerical values. Examples include:

- House price prediction
- Stock price forecasting
- Temperature prediction

Common Supervised Learning Algorithms

- Linear Regression
- Logistic Regression
- Support Vector Machines (SVM)
- Decision Trees
- Random Forests
- Neural Networks

1.2.2 UNSUPERVISED LEARNING

Definition: Unsupervised learning is a type of machine learning where the algorithm learns patterns from unlabelled data without predefined categories.



Unlike supervised learning, there are no target labels. The algorithm must discover the underlying structure in the data on its own.

Clustering

Clustering groups similar data points together based on their features. Examples include:

- Customer segmentation for marketing
- Document categorization
- Image segmentation

Common clustering algorithms:

- K-Means
- Hierarchical Clustering
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Dimensionality Reduction

Dimensionality reduction reduces the number of features while preserving important information. This is useful for:

- Visualization of high-dimensional data
- Reducing computational complexity
- Removing noise and redundant features

Common techniques:

- Principal Component Analysis (PCA)
- t-SNE (t-Distributed Stochastic Neighbour Embedding)
- Autoencoders

Anomaly Detection

Anomaly detection identifies unusual patterns that do not conform to expected behaviour. Applications include:

- Fraud detection
- Network intrusion detection
- Medical anomaly detection



1.2.3 REINFORCEMENT LEARNING

Definition: Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative reward.

Key components of reinforcement learning:

- **Agent:** The learner or decision-maker
- **Environment:** The world the agent interacts with
- **State:** The current situation of the agent
- **Action:** Choices available to the agent
- **Reward:** Feedback signal indicating success or failure
- **Policy:** The strategy the agent uses to select actions

The agent learns through trial and error, receiving rewards or penalties for its actions.

Over time, it develops a policy that maximizes expected rewards.

Applications:

- Game playing (Chess, Go, video games)
- Robotics and autonomous systems
- Resource management
- Recommendation systems

1.2.4 THE FOUR SECTORS OF QML

QML research is extensive and can be broadly divided into four primary sectors, each defined by the nature of the computing resources (whether the computing device is quantum **Q** or classical **C**) and the type of data involved (whether generated by a quantum **Q** or classical **C** system).

[IMAGE: Figure 1.2 - Different research directions in QML]

Source: Du et al., "Quantum Machine Learning: A Hands-on Tutorial," 2025, p. 10

CC Sector (Classical-Classical)



The CC sector refers to classical data processed on classical systems, representing traditional machine learning. Here, classical ML algorithms run on classical processors (e.g., CPUs and GPUs) and are applied to classical datasets.

Example: Using neural networks to classify images of cats and dogs.

CQ Sector (Classical-Quantum)

The CQ sector involves using classical ML algorithms on classical processors to analyze quantum data collected from quantum systems.

Examples:

- Applying classical neural networks to classify quantum states
- Estimating properties of quantum systems from measurement data
- Employing classical regression models to predict outcomes of quantum experiments

QC Sector (Quantum-Classical)

The QC sector involves developing QML algorithms that run on quantum processors (QPUs) to process classical data. In this context, quantum computing resources are leveraged to enhance or accelerate the analysis of classical datasets.

Examples:

- Applying QML algorithms, such as quantum neural networks and quantum kernels
- Improving pattern recognition in image analysis

QQ Sector (Quantum-Quantum)

The QQ sector involves developing QML algorithms executed on QPUs to process quantum data. In this context, quantum computing resources are leveraged to reduce the computational cost of analyzing and understanding complex quantum systems.

Examples:

- Using quantum neural networks for quantum state classification
- Applying quantum-enhanced algorithms to simulate quantum many-body systems

***Note:** The primary focus of this course is on the **QC** and **QQ** sectors.*



1.2.5 INTRODUCTION TO KERNEL METHODS

The Problem of Non-Linear Data

In many machine learning tasks, particularly in classification, the goal is to find a decision boundary that best separates different classes of data. When the data is linearly separable, this boundary can be a straight line, plane, or hyperplane. However, in real-world scenarios, data is often *not* linearly separable.

[IMAGE: Figure 3.1 - Various distributions of data points]

Source: Du et al., "Quantum Machine Learning: A Hands-on Tutorial," 2025, p. 68

The left panel shows cases where data points can be separated by a straight line. The middle panel shows data that cannot be separated by a straight line. The right panel shows that the kernel function could map the linearly inseparable data points into high-dimensional linearly separable data points.

Traditional linear models struggle with such non-linear data because they are inherently limited to creating only linear decision boundaries. This limitation highlights the need for more flexible approaches.

Feature Mapping

To address the challenge of non-linear data, one effective strategy is to transform the input data into a higher-dimensional space where the data may become linearly separable. This transformation is known as *feature mapping*, denoted by:

$$\phi : x \rightarrow \phi(x) \in \mathbb{R}^D$$

where the original input space X is mapped to a higher-dimensional feature space $\mathbb{R}^D$ with $D \geq d$. The idea is that, in this higher-dimensional space, complex patterns in the original data can be more easily identified using linear models.

The Kernel Trick

However, explicitly computing the feature map $\phi(x)$ can be computationally expensive, especially if the feature space is high-dimensional or even infinite-dimensional. Fortunately, many machine learning algorithms for tasks like classification or regression depend primarily on the inner product between data points.

In the feature space, this inner product is given by $\langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$.



To circumvent the computational cost of explicitly calculating the feature map, we can use a *kernel function* $k(x^{(i)}, x^{(j)})$ defined as:

$$k(x^{(i)}, x^{(j)}) = \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle$$

This allows us to compute the inner product in the higher-dimensional feature space *indirectly*, without ever having to compute $\phi(x)$ explicitly. This approach is commonly known as the **kernel trick**.

By using the kernel function directly within algorithms, we avoid the computational overhead of working in a high-dimensional space. The collection of kernel values for a dataset forms the *kernel matrix* (or Gram matrix), where each entry is given by $k(x^{(i)}, x^{(j)})$. This matrix is central to many kernel-based algorithms, as it captures the pairwise similarities between all training data points.

[IMAGE: Figure 3.2 - The paradigm of classical and quantum kernels]

Source: Du et al., "Quantum Machine Learning: A Hands-on Tutorial," 2025, p. 76

Common Kernel Functions

1. Polynomial Kernel

A polynomial kernel of degree $m \in \mathbb{N}$ with a constant $c > 0$ is defined as:

$$k(x, x') = (x \cdot x' + c)^m, \forall x, x' \in \mathbb{R}^d$$

This kernel maps the input space to a higher-dimensional space of dimension $C(d+m, m)$. For instance, in a two-dimensional input space ($d = 2$) and with $m = 2$, the kernel corresponds to an inner product in a higher-dimensional space of dimension 6.

2. Gaussian Kernel (Radial Basis Function - RBF)

The Gaussian kernel (or Radial Basis Function, RBF) is one of the most widely used kernels, defined as:

$$k(x, x') = \exp(-||x - x'||^2 / 2\sigma^2)$$

where $\sigma > 0$ controls the width of the Gaussian function.

Gaussian kernels are particularly effective in capturing complex non-linear patterns due to their infinite-dimensional feature space.

3. Sigmoid Kernel

The sigmoid kernel over $\mathbb{R}^d$ is defined as:



UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING

$$k(x, x') = \tanh(a \cdot (x \cdot x') + b), \forall x, x' \in \mathbb{R}^d$$

where $a, b > 0$ are constants, and $\tanh(c) = (e^c - e^{-c}) / (e^c + e^{-c})$ is the hyperbolic tangent function.

This kernel is related to neural networks, as it resembles the activation function commonly used in multi-layer perceptrons.

Support Vector Machines (SVMs)

Support Vector Machines are a well-known algorithm that heavily relies on kernel methods and are primarily used for classification tasks. The objective of an SVM is to identify the optimal hyperplane that separates data points from different classes with the *maximum margin*. The margin is defined as the distance between the hyperplane and the closest data points from each class, known as support vectors.

SVMs can be applied to both linear and non-linear classification problems. For non-linear cases, kernel functions are employed to map the data into higher-dimensional spaces, enabling the separation of data that is not linearly separable in the original space.

1.2.6 APPLICATIONS OF MACHINE LEARNING

Machine learning has found applications across virtually every industry:

Image Recognition

- Facial recognition systems
- Medical image analysis
- Autonomous vehicle perception

Natural Language Processing

- Language translation
- Sentiment analysis
- Chatbots and virtual assistants

Healthcare and Drug Discovery

- Disease diagnosis
- Drug interaction prediction



UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING

- Personalized medicine

Finance and Risk Assessment

- Fraud detection
- Credit scoring
- Algorithmic trading

Introduction to Quantum Applications

As quantum hardware continues to mature, QML applications are emerging in:

- Chemistry: Molecular simulation and drug discovery
- Computer vision: Enhanced pattern recognition
- Power systems: Optimization of energy distribution
- Logistics: Route optimization and resource allocation
- Finance: Portfolio optimization and risk analysis
- Healthcare: Medical imaging and genomics

1.3 SCI-KIT LEARN FRAMEWORK

1.3.1 INTRODUCTION TO SCIKIT-LEARN

What is Scikit-learn?

Scikit-learn is an open-source Python library that provides simple and efficient tools for data mining and data analysis. Built on NumPy, SciPy, and matplotlib, it is one of the most popular machine learning libraries in the Python ecosystem.

Key Features:

- Simple and consistent API
- Comprehensive documentation
- Wide range of supervised and unsupervised learning algorithms
- Tools for model selection and evaluation
- Data preprocessing utilities

Installation



To install Scikit-learn, use pip:

```
pip install scikit-learn
```

Or with conda:

```
conda install scikit-learn
```

Library Structure Overview

Scikit-learn is organized into several modules:

- `sklearn.datasets`: Dataset loading utilities
- `sklearn.preprocessing`: Data preprocessing tools
- `sklearn.model_selection`: Model selection and evaluation
- `sklearn.linear_model`: Linear models
- `sklearn.tree`: Decision tree models
- `sklearn.ensemble`: Ensemble methods
- `sklearn.cluster`: Clustering algorithms
- `sklearn.metrics`: Evaluation metrics

1.3.2 DATA HANDLING IN SCIKIT-LEARN

Loading Built-in Datasets

Scikit-learn provides several built-in datasets for practice:

```
from sklearn.datasets import load_iris, load_digits,
load_wine

# Load the Iris dataset
iris = load_iris()
X = iris.data # Features
y = iris.target # Labels

print(f"Feature shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"Feature names: {iris.feature_names}")
print(f"Target names: {iris.target_names}")
```



Data Preprocessing

Data often needs to be preprocessed before training:

```
from sklearn.preprocessing import StandardScaler,
MinMaxScaler

# Standardization (zero mean, unit variance)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Min-Max scaling (scale to [0, 1] range)
minmax_scaler = MinMaxScaler()
X_minmax = minmax_scaler.fit_transform(X)
```

Train-Test Split

Always split your data into training and testing sets:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2, # 20% for testing
    random_state=42 # For reproducibility
)

print(f"Training set size: {len(X_train)}")
print(f"Test set size: {len(X_test)}")
```

1.3.3 BUILDING YOUR FIRST MODEL

The Estimator API

All Scikit-learn models follow the same interface (Estimator API):

1. **Instantiate** the model with parameters
2. **Fit** the model to training data
3. **Predict** on new data
4. **Score** to evaluate performance

Fit-Predict Pattern



```
from sklearn.linear_model import LogisticRegression

# Step 1: Instantiate the model
model = LogisticRegression(max_iter=200)

# Step 2: Fit the model to training data
model.fit(X_train, y_train)

# Step 3: Make predictions
y_pred = model.predict(X_test)

# Step 4: Evaluate
accuracy = model.score(X_test, y_test)
print(f"Accuracy: {accuracy:.2f}")
```

Simple Classification Example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load data
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Evaluate
print(f"Training accuracy: {model.score(X_train, y_train):.2f}")
print(f"Test accuracy: {model.score(X_test, y_test):.2f}")
```



1.3.4 MODEL EVALUATION

Accuracy and Error Metrics

```
from sklearn.metrics import accuracy_score,
precision_score, recall_score, f1_score

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred,
average='weighted')
recall = recall_score(y_test, y_pred,
average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
```

Confusion Matrix

A confusion matrix shows the distribution of predictions:

```
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

# Visualize (optional)
# disp = ConfusionMatrixDisplay(confusion_matrix=cm)
# disp.plot()
# plt.show()
```

Cross-Validation

Cross-validation provides a more robust estimate of model performance:



```
from sklearn.model_selection import cross_val_score

# 5-fold cross-validation
scores = cross_val_score(model, X, y, cv=5)

print(f"CV Scores: {scores}")
print(f"Mean CV Score: {scores.mean():.2f} (+/-
{scores.std() * 2:.2f})")
```

1.3.5 PRACTICAL EXAMPLE: CLASSIFICATION TASK

Complete Workflow: Iris Dataset Classification

```
# Import libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split,
cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
confusion_matrix

# 1. Load data
iris = load_iris()
X, y = iris.data, iris.target

print("Dataset loaded:")
print(f"  Samples: {X.shape[0]}")
print(f"  Features: {X.shape[1]}")
print(f"  Classes: {len(set(y))}")

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Preprocess (scale features)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```



```
# 4. Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train_scaled, y_train)

# 5. Evaluate
y_pred = model.predict(X_test_scaled)

print("\nClassification Report:")
print(classification_report(y_test, y_pred,
                             target_names=iris.target_names))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# 6. Cross-validation
cv_scores = cross_val_score(model,
                             scaler.fit_transform(X),
                             y, cv=5)
print(f"\n5-Fold CV Accuracy: {cv_scores.mean():.2f}
      (+/- {cv_scores.std()*2:.2f})")
```

Expected Output:

```
Dataset loaded:
  Samples: 150
  Features: 4
  Classes: 3

Classification Report:
              precision    recall  f1-score   support

   setosa         1.00        1.00        1.00         10
  versicolor      1.00        1.00        1.00         10
   virginica      1.00        1.00        1.00         10

 accuracy                   1.00         30
  macro avg           1.00        1.00        1.00         30
 weighted avg          1.00        1.00        1.00         30

Confusion Matrix:
```



```
[[10  0  0]
 [ 0 10  0]
 [ 0  0 10]]
```

```
5-Fold CV Accuracy: 0.97 (+/- 0.06)
```

1.4 SUMMARY

In this unit, we explored the foundations of classical machine learning that prepare us for understanding quantum machine learning:

1. **Machine Learning Fundamentals:** We defined machine learning as a field focused on building systems that learn from data rather than explicit programming. We traced the evolution from CPUs to GPUs and understood why quantum computing represents the next frontier.
2. **Types of Learning:** We distinguished between supervised learning (learning with labels), unsupervised learning (discovering patterns without labels), and reinforcement learning (learning through interaction). We also introduced the four sectors of QML: CC, CQ, QC, and QQ.
3. **Kernel Methods:** We learned how kernel methods address the challenge of non-linear data by implicitly mapping data to higher-dimensional spaces using the kernel trick. Common kernels include polynomial, Gaussian (RBF), and sigmoid kernels.
4. **Scikit-learn Framework:** We gained hands-on experience with Python's Scikit-learn library, learning the fit-predict pattern, data preprocessing, and model evaluation techniques.

Key Concepts Review:

- **Machine Learning:** Systems that learn from data to make predictions
- **Supervised Learning:** Learning from labelled examples
- **Unsupervised Learning:** Finding patterns in unlabelled data
- **Reinforcement Learning:** Learning through rewards and penalties
- **Feature Mapping:** Transforming data to higher-dimensional space



UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING

- **Kernel Trick:** Computing inner products implicitly in feature space
- **Estimator API:** Scikit-learn's fit-predict interface

Connection to Quantum Machine Learning:

The concepts covered in this unit form the essential foundation for quantum machine learning:

- **Kernel methods** directly connect to **quantum kernel methods** (Unit 7), where quantum circuits create feature maps in exponentially large Hilbert spaces
- **Support Vector Machines** have quantum counterparts (Quantum SVM) that leverage quantum feature spaces
- **Neural networks** (briefly introduced) will be extended to **Quantum Neural Networks** (Unit 11)
- Understanding the **four QML sectors** helps contextualize where quantum approaches can provide advantages

In the upcoming units, you will learn how quantum computing fundamentals (Unit 2) and quantum programming platforms (Unit 3) enable the quantum versions of these classical algorithms.

1.5 MIND MAP

[Mind map to be added]

1.6 GLOSSARY

Machine Learning (ML) — A branch of AI focused on building systems that learn from data to make predictions or decisions.

Supervised Learning — Learning paradigm where the algorithm learns from labelled training data.

Unsupervised Learning — Learning paradigm where the algorithm discovers patterns in unlabelled data.



Reinforcement Learning — Learning paradigm where an agent learns through interaction with an environment to maximize rewards.

Classification — Supervised learning task that predicts categorical labels.

Regression — Supervised learning task that predicts continuous numerical values.

Clustering — Unsupervised learning task that groups similar data points together.

Kernel Function — A function that computes the inner product of data points in a higher-dimensional feature space.

Feature Mapping (ϕ) — A transformation that maps data from input space to a higher-dimensional feature space.

Kernel Trick — Technique for computing inner products in high-dimensional spaces without explicit feature mapping.

Support Vector Machine (SVM) — Algorithm that finds the optimal hyperplane separating classes with maximum margin.

Quantum Machine Learning (QML) — Field exploring learning algorithms executed on quantum computers with potential advantages over classical approaches.

Qubit — Quantum bit; fundamental unit of quantum information that can exist in superposition of states.

BQP — Bounded-Error Quantum Polynomial Time; complexity class of problems solvable efficiently by quantum computers.

NISQ — Noisy Intermediate-Scale Quantum; current era of quantum computing with limited qubits and error-prone gates.

Quantum Advantage — Demonstrated improvement in efficiency (runtime, sample complexity) of quantum algorithms over classical algorithms.

Quantum Utility — Quantum computations yielding reliable solutions beyond brute-force classical methods.



1.7 SELF-ASSESSMENT QUESTIONS

1. Which of the following best describes the fundamental difference between traditional programming and machine learning?
 - a) Traditional programming uses compiled languages, while machine learning uses interpreted languages
 - b) In traditional programming, rules are explicitly coded by the developer, whereas in machine learning the system learns rules from data
 - c) Machine learning requires more storage than traditional programming
 - d) Traditional programming cannot process numerical data, while machine learning can
2. A researcher collects sensor data from an industrial plant without any labels and wants to identify unusual patterns that could indicate equipment failure. Which type of learning is most appropriate for this task?
 - a) Supervised learning — classification
 - b) Supervised learning — regression
 - c) Unsupervised learning — anomaly detection
 - d) Reinforcement learning
3. In the context of supervised learning, what distinguishes a classification task from a regression task?
 - a) Classification uses more training data than regression
 - b) Classification predicts categorical labels, while regression predicts continuous numerical values
 - c) Classification works only with images, while regression works only with tabular data
 - d) Regression requires labelled data, but classification does not
4. The kernel trick is valuable in machine learning because it:
 - a) Reduces the number of training samples required for convergence
 - b) Allows computation of inner products in a high-dimensional feature space without explicitly computing the feature map
 - c) Eliminates the need for a training phase in SVM algorithms



- d) Converts unsupervised learning problems into supervised ones
5. A QML researcher develops a quantum neural network that runs on a quantum processor to classify images from a standard (classical) dataset. Which QML sector does this work belong to?
- a) CC (Classical-Classical)
 - b) CQ (Classical-Quantum)
 - c) QC (Quantum-Classical)
 - d) QQ (Quantum-Quantum)
6. Which statement best explains why an N-qubit system can encode exponentially more information than N classical bits?
- a) Qubits are physically larger than classical bits and store more information per unit
 - b) An N-qubit system is described by a 2^N -dimensional vector due to the tensor product rule, whereas N classical bits represent only one of 2^N possible states at a time
 - c) Quantum computers operate at higher clock speeds than classical computers
 - d) Classical bits can only represent integers, while qubits can represent real numbers
7. In Scikit-learn's Estimator API, what is the correct sequence of operations for training a model and obtaining predictions?
- a) predict → fit → score
 - b) score → fit → predict
 - c) fit → predict → score
 - d) predict → score → fit
8. Why does k-fold cross-validation generally provide a more reliable performance estimate than a single train-test split?
- a) It uses a larger test set than a single split
 - b) It trains the model on the entire dataset without any splitting
 - c) It evaluates the model across multiple different data partitions, reducing the impact of any particular split
 - d) It eliminates the need for a separate test set entirely



UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING

9. Given a binary classifier's confusion matrix $\begin{bmatrix} 45 & 5 \\ 10 & 40 \end{bmatrix}$, what is the overall accuracy of the model?
- a) 75%
 - b) 80%
 - c) 85%
 - d) 90%
10. A machine learning engineer needs a kernel function that can capture highly complex, non-linear decision boundaries. Which kernel would be most suitable, and why?
- a) Linear kernel, because it produces the simplest and most interpretable boundaries
 - b) Polynomial kernel, because it always maps to an infinite-dimensional space
 - c) Gaussian (RBF) kernel, because its infinite-dimensional feature space enables it to capture complex non-linear patterns
 - d) Sigmoid kernel, because it directly mimics quantum circuit behaviour

1.8 TERMINAL QUESTIONS

SHORT-ANSWER QUESTIONS

1. Distinguish between classification and regression in supervised learning, providing one real-world example of each.
2. Explain the concept of the kernel trick and its computational advantage over explicit feature mapping.
3. Describe the four sectors of Quantum Machine Learning (CC, CQ, QC, QQ) and provide one example for each sector.
4. What is the Estimator API pattern in Scikit-learn? Describe its steps and explain why a consistent API design matters.
5. Explain the purpose of k-fold cross-validation and how it provides a more reliable performance estimate than a single train-test split.



6. What is a confusion matrix? Describe how accuracy, precision, and recall can be derived from it.

LONG-ANSWER QUESTIONS

1. Compare and contrast the three main types of learning paradigms—supervised, unsupervised, and reinforcement learning. For each paradigm, describe its key characteristics, the nature of the training data, at least two common algorithms, and two real-world applications.
2. Explain kernel methods in detail, covering the problem of non-linear data, feature mapping, the kernel trick, at least two common kernel functions (with their formulas), and how Support Vector Machines use kernels for classification.
3. Describe the complete machine learning workflow from data collection to model evaluation. For each step, explain what it involves and how the Scikit-learn library supports it with specific modules or functions.
4. Discuss the relationship between classical machine learning and quantum machine learning. Explain how quantum computing can potentially enhance machine learning, identify which QML sectors are the primary focus of current research, and describe the role of classical ML knowledge as a foundation for QML.

1.9 ANSWERS

1.9.1 SELF-ASSESSMENT ANSWERS

1. **(b)** In traditional programming, rules are explicitly coded by the developer, whereas in machine learning the system learns rules from data. Traditional programming requires explicit instructions for every scenario, while ML systems learn patterns from examples and adapt their behaviour.
2. **(c)** Unsupervised learning — anomaly detection. Since the data is unlabelled and the goal is to identify unusual patterns, this is an unsupervised anomaly detection task. Supervised learning would require labelled examples of failures.



3. **(b)** Classification predicts categorical labels (e.g., spam or not spam), while regression predicts continuous numerical values (e.g., house prices). Both are supervised learning approaches that require labelled training data.
4. **(b)** The kernel trick allows us to compute inner products in a high-dimensional feature space indirectly, using a kernel function $k(x, x') = \langle \phi(x), \phi(x') \rangle$, without ever explicitly computing the feature map $\phi(x)$. This avoids the computational overhead of working in high-dimensional spaces.
5. **(c)** QC (Quantum-Classical). The QC sector involves QML algorithms running on quantum processors (QPUs) to process classical data. Since the researcher uses a quantum neural network on a quantum processor for a classical image dataset, this falls in the QC sector.
6. **(b)** Due to the tensor product rule governing qubits, an N-qubit system is described by a 2^N -dimensional complex vector, allowing quantum systems to encode information exponentially with N. In contrast, N classical bits follow the Cartesian product rule and represent only one of 2^N possible states at any given time.
7. **(c)** fit $\rightarrow$ predict $\rightarrow$ score. The Estimator API follows a consistent pattern: first the model is fitted (trained) on training data, then predictions are made on new data, and finally the model's performance is evaluated using a scoring method.
8. **(c)** Cross-validation evaluates the model across multiple different data partitions. In k-fold CV, the data is split into k subsets, and the model is trained and tested k times on different combinations, reducing the dependence on any single split and giving a more stable performance estimate.
9. **(c)** 85%. Accuracy = (True Positives + True Negatives) / Total = $(45 + 40) / (45 + 5 + 10 + 40) = 85/100 = 85\%$.
10. **(c)** The Gaussian (RBF) kernel is most suitable because it maps data into an infinite-dimensional feature space, making it particularly effective at capturing complex non-linear patterns. The polynomial kernel maps to a finite-dimensional space, and the linear kernel creates only linear boundaries.



1.9.2 TERMINAL QUESTION ANSWERS

1. Classification predicts categorical (discrete) labels, while regression predicts continuous numerical values. In classification, the model assigns inputs to predefined classes separated by decision boundaries. For example, an email spam filter classifies each message as "spam" or "not spam" based on features such as word frequency and sender information. In regression, the model outputs a continuous number. For example, a house price prediction model estimates a numerical sale price based on features like square footage, location, and number of bedrooms. Both are supervised learning tasks requiring labelled training data, but they differ in the nature of their output space—categorical versus continuous.
2. The kernel trick allows machine learning algorithms to compute inner products in a high-dimensional feature space without explicitly computing the feature map $\phi(x)$. Instead of transforming each data point into a higher-dimensional space (which can be computationally prohibitive when the space is very high-dimensional or infinite-dimensional), a kernel function $k(x, x') = \langle \phi(x), \phi(x') \rangle$ directly computes the inner product. This is computationally advantageous because it avoids the cost of explicitly calculating and storing the high-dimensional feature vectors. For example, the Gaussian (RBF) kernel maps to an infinite-dimensional space, yet its kernel value is computed with a simple exponential formula.
3. The four QML sectors are defined by the type of computing resources and data involved. **CC (Classical-Classical)**: Classical algorithms process classical data on classical processors, representing traditional ML—for example, using neural networks to classify images. **CQ (Classical-Quantum)**: Classical algorithms analyse quantum data, such as using classical regression to predict outcomes of quantum experiments. **QC (Quantum-Classical)**: Quantum algorithms process classical data on quantum processors, such as applying quantum neural networks to classical image datasets. **QQ (Quantum-Quantum)**: Quantum algorithms process quantum data, such as using quantum neural networks for quantum state classification.



4. The Estimator API is Scikit-learn's consistent interface pattern that all models follow. It consists of four steps: (1) **Instantiate** the model with hyperparameters, (2) **Fit** the model to training data using `model.fit(X_train, y_train)`, (3) **Predict** on new data using `model.predict(X_test)`, and (4) **Score** to evaluate performance using `model.score(X_test, y_test)`. This consistent API matters because it allows users to swap algorithms easily without changing the surrounding code, simplifies learning new models, and enables standardised model comparison and pipeline construction.
5. K-fold cross-validation divides the dataset into k equal subsets (folds). The model is trained k times, each time using k-1 folds for training and the remaining fold for testing. The final performance metric is the average across all k iterations. This provides a more reliable estimate than a single train-test split because it reduces dependency on any particular data partition. A single split may produce optimistic or pessimistic results depending on which data points fall in the test set. Cross-validation averages over multiple partitions, yielding a more stable and representative assessment of how the model will perform on unseen data.
6. A confusion matrix is a table that summarises the performance of a classification model by comparing predicted labels against true labels. For binary classification, it contains four values: True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). From these values: **Accuracy** = $(TP + TN) / (TP + TN + FP + FN)$, measuring overall correctness. **Precision** = $TP / (TP + FP)$, measuring the proportion of positive predictions that are correct. **Recall** = $TP / (TP + FN)$, measuring the proportion of actual positives that are correctly identified. Each metric highlights a different aspect of model performance.
7. The three main learning paradigms differ fundamentally in their data requirements, objectives, and applications.

Supervised learning uses labelled data where each training example has input features paired with a known target label. The goal is to learn a mapping from inputs to outputs. Common algorithms include Linear Regression (for continuous targets), Support Vector



Machines (for classification), Decision Trees, and Neural Networks. Applications include email spam detection, medical diagnosis, house price prediction, and image recognition.

Unsupervised learning uses unlabelled data and aims to discover hidden structure or patterns. No target labels are provided—the algorithm must find patterns autonomously. Common algorithms include K-Means and DBSCAN (for clustering), and Principal Component Analysis (for dimensionality reduction). Applications include customer segmentation, anomaly detection, document categorisation, and data visualisation.

Reinforcement learning involves an agent interacting with an environment, learning through trial and error to maximise cumulative reward. Training data comes from the agent's own experience rather than a static dataset. Key concepts include states, actions, rewards, and policies. Algorithms include Q-learning and policy gradient methods. Applications include game playing (AlphaGo), robotics, autonomous vehicles, and recommendation systems.

The paradigms are complementary: supervised learning excels when labelled data is available, unsupervised learning discovers patterns in unlabelled data, and reinforcement learning handles sequential decision-making tasks.

8. Kernel methods address the fundamental challenge of non-linear data in machine learning. When data is not linearly separable—meaning no straight line or hyperplane can separate different classes—linear models fail.

Feature mapping transforms data into a higher-dimensional space $\phi: x \rightarrow \phi(x) \in \mathbb{R}^D$ where $D \geq d$, hoping that data becomes linearly separable in the new space. However, explicitly computing $\phi(x)$ is expensive when D is large or infinite.

The **kernel trick** bypasses this by defining a kernel function $k(x, x') = \langle \phi(x), \phi(x') \rangle$ that computes the inner product in the high-dimensional space directly. Common kernel functions include: the **Polynomial kernel** $k(x, x') = (x \cdot x' + c)^m$, which maps to a finite-dimensional space of dimension $C(d+m, m)$; and the **Gaussian (RBF) kernel** $k(x, x') = \exp(-\|x - x'\|^2 / 2\sigma^2)$, which maps to an infinite-dimensional space and excels at capturing complex non-linear patterns.



Support Vector Machines heavily rely on kernels. An SVM finds the optimal hyperplane with maximum margin between classes. Support vectors are the closest data points to the boundary. For non-linear data, the kernel function enables the SVM to find complex decision boundaries in the original input space while actually computing a linear separator in the high-dimensional feature space.

9. The complete ML workflow has six major steps, each supported by Scikit-learn.

Data Collection involves gathering relevant data. Scikit-learn provides built-in datasets via `sklearn.datasets` (e.g., `load_iris()`, `load_digits()`) for practice and prototyping.

Data Preparation includes cleaning, handling missing values, and formatting. Scikit-learn's `sklearn.preprocessing` module offers `StandardScaler` (for zero-mean, unit-variance normalisation), `MinMaxScaler` (for scaling to [0,1]), and other transformers.

Data Splitting separates data into training and testing sets. The `train_test_split()` function from `sklearn.model_selection` handles this, supporting stratified splitting and random seed control for reproducibility.

Model Selection and Training involves choosing an algorithm and fitting it. All models follow the Estimator API: instantiate with hyperparameters, then call `model.fit(X_train, y_train)`. Available models span `sklearn.linear_model`, `sklearn.tree`, `sklearn.ensemble`, and more.

Prediction uses `model.predict(X_test)` to generate outputs for new data, following the same interface regardless of algorithm.

Evaluation uses metrics from `sklearn.metrics` including `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, and `confusion_matrix`. Cross-validation via `cross_val_score()` provides robust performance estimates across multiple data splits.

10. Classical ML provides the essential foundation for understanding QML. The mathematical concepts underlying classical ML—such as kernel methods,



optimisation, linear algebra, and probability—transfer directly to the quantum domain.

Quantum computing can enhance ML through several mechanisms. The exponential state space of qubits (an N-qubit system represents a 2^N -dimensional vector) enables encoding and processing large datasets more efficiently. Quantum kernels can potentially compute similarity measures in feature spaces that are classically intractable, enabling new types of non-linear classifiers. Quantum algorithms like the HHL algorithm offer potential speedups for linear algebra operations central to ML.

The QML research landscape is organised into four sectors. The **QC sector** (quantum algorithms on classical data) and **QQ sector** (quantum algorithms on quantum data) are the primary focus of current research. The QC sector applies quantum neural networks and quantum kernels to classical datasets, while the QQ sector uses quantum ML for tasks like quantum state classification and simulation.

Classical ML knowledge is indispensable for QML because: quantum models use analogous training procedures (loss functions, gradient descent), evaluation metrics are shared (accuracy, cross-validation), and frameworks like PennyLane integrate classical ML libraries (PyTorch, TensorFlow) with quantum circuits. Understanding classical ML fundamentals ensures researchers can effectively design, train, and evaluate quantum models.

1.10 REFERENCES

1. Du, Y., Wang, X., Guo, N., Yu, Z., Qian, Y., Zhang, K., Hsieh, M.-H., Rebentrost, P., & Tao, D. (2025). Quantum Machine Learning: A Hands-on Tutorial for Machine Learning Practitioners and Researchers. *arXiv preprint arXiv:2502.01146*.
2. Pattanayak, S. (2021). *Quantum Machine Learning with Python: Using Cirq from Google Research and IBM Qiskit*. Apress.
3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

UNIT 01: FOUNDATIONS OF CLASSICAL MACHINE LEARNING



4. Nielsen, M. A., & Chuang, I. L. (2011). *Quantum Computation and Quantum Information* (10th anniversary ed.). Cambridge University Press.
5. Mohri, M., Rostamizadeh, A., & Talwalkar, A. (2018). *Foundations of Machine Learning* (2nd ed.). MIT Press.

End of Unit 1